

Homework 2

Due date: Thursday, October 1, 2026, 11:59 PM on Gradescope

Overview:

In this assignment, you will "defuse" a binary bomb: a compiled program made of six phases (plus a hidden bonus phase). Each phase asks for a single line of input. Enter the correct input to defuse the phase; enter the wrong input and the bomb explodes.

Phases are compiled into the binary. Your job is to reverse-engineer the executable with a debugger and a disassembler to figure out what each phase wants.

Your bomb is a 64-bit RISC-V executable and is run in a QEMU environment with Docker.

Background:

A binary bomb is a program made of six phases. Each phase tests a different concept in machine-level programming:

1. Strings
2. Loops
3. Conditionals
4. Function calls & the stack
5. Pointers
6. Linked Lists

What You'll Practice:

- Reading and tracing RISC-V assembly: calling conventions, branches, loads/stores.
- Using a debugger with breakpoints, stepping, inspecting
- Handling compiled binaries

Setup:

1. Go to [Classroom 50](https://classroom50.org) (link: <https://classroom50.org>) and sign in with your Github account.
2. Click the "Accept Assignment" button on HW2.
 - a. Or use this link:
<https://classroom50.org/cmpe120/cmpe-120-fa26/assignments/hw2/accept>
3. Click the "Open Repository" button.
4. A new private repo should be created with the following files in it:

 .classroom50.yaml	[Classroom 50] Initialize .classroom50.yaml and autograde...	now
 README.md	Initial commit	now
 your-github-username.txt	Initial commit	now

a.

5. Open the repo in your text editor of choice (e.g., VSCode). Then follow these instructions:

a. If you haven't already done so, install github cli tool:

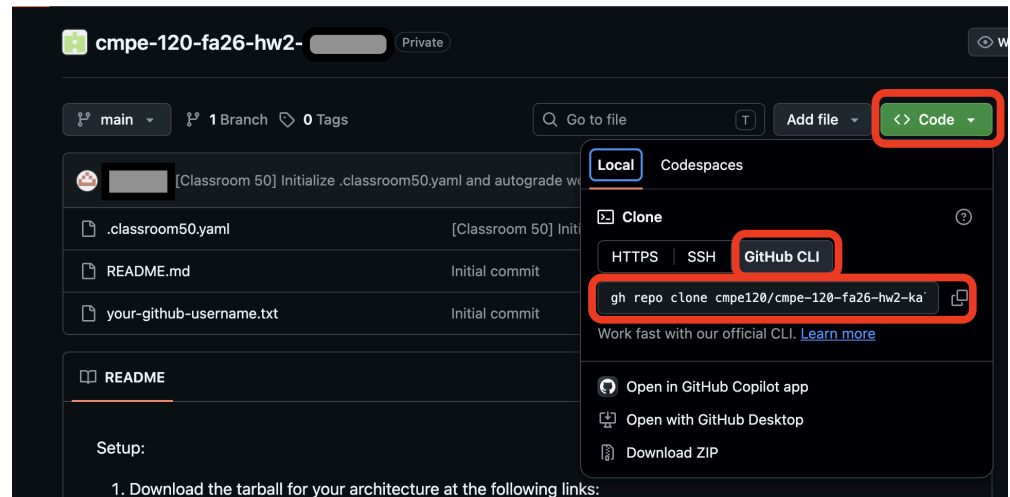
i. On Windows, go to Command Prompt and run:

1. `winget install --id GitHub.cli`

ii. On Mac, make sure you have homebrew installed (if not, install instructions here: <https://brew.sh/>). Then, go to Terminal and run:

1. `brew install gh`

b. Go to the green code button in the top right corner and change the tab to "Github-CLI"



i.

ii. Copy the CLI command and run it in Command Prompt/Terminal.

iii. Open your repo in your text editor.

6. Follow the instructions in the README.md to set up your Docker environment. Viewing the README on the browser is recommended.

Gradescope Submission:

You submit one text file through Gradescope using the GitHub option. In Gradescope, open the assignment and choose GitHub as the submission method. Authenticate with the GitHub account that has your repository, then select the repository and branch containing your .txt file. If the Gradescope is unable to download the repo, then you should download your github repo as a zip and upload that to gradescope instead. Downloading instructions are here:

<https://docs.github.com/en/repositories/working-with-files/using-files/downloading-files-from-github>

Required File Format:

1. Name your file exactly: <your-github-username>.txt
2. Line 1 - Your student ID
3. Line 2 onward - One answer per line (A phase that expects several values takes them on a single line separated by spaces.)

Example:

100000001

<phase-1 answer>

<phase-2 answer>

<phase-3 answer>

<phase-4 answer>

<phase-5 answer>

<phase-6 answer>

Gradescope Results:

After submission, the autograder reports a per-phase score out of 100 points:

Phase 1: 15

Phase 2: 15

Phase 3: 15

Phase 4: 15

Phase 5: 20

Phase 6: 20

Result: 100

Secret Phase: +5 extra credit

Note: Every bomb is unique, so a copied answer earns zero: your submission is always validated against your bomb. Some common results you could see are:

1. A numeric score, the points you earned (see above).
2. “defused” / “not defused” per phase, whether each phase’s answer was accepted.
3. “Could not map the submission to a bomb”, your filename doesn’t match your GitHub username.
4. “Student ID mismatch”, line 1 of your file doesn’t match the student ID on file for your username.
5. Score 0 with a short message, usually a filename/ID/format problem (see above), not a grading failure.

Useful GDB Commands:

break <func/line>: Set a breakpoint

run: Start the program

cont: Continue execution until the next breakpoint

next (n) / nexti (ni): Execute the next line/instruction without stepping into a function

stepi (si): Execute the next line/instruction and step into a function

finish: Execute through the end of the current function

print <var>: Print the value of a variable

where: Print the call stack

layout asm: Show the assembly code in a split window

layout src: Show the source code in a split window

layout split: Show source code and assembly in split windows

focus <cmd/src/asm>: Change keyboard control between split windows

More information and helpful gdb commands can be found here:

<https://shakti.org.in/docs/RISC-V-GDB-tutorial.pdf>

Troubleshooting:

- get-bomb says “No bomb found” means you typed the wrong username; use your exact GitHub username.
- “Unable to find image 'bomblab-student:arm64' locally”: you may have to run docker login
- input goes to the wrong place while debugging: Make sure QEMU is running in the foreground in Terminal 1 and GDB is running in a second terminal attached to the same Docker container. Bomb answers should be typed in Terminal 1; GDB commands should be typed in Terminal 2.
- docker run can't find your work/ folder, the -v mount path must be an absolute path (on Windows, a literal C:/... path).
- Qemu error: bind: Address already in use / could not open gdbserver on 1234. This usually means another QEMU process is already using port 1234. Do not start another QEMU instance. Return to the terminal running the existing QEMU process and stop it cleanly, then restart QEMU.
- GDB warnings about shared libraries or file transfer: You may see warnings related to downloading shared-library files, the dynamic linker, or missing debugging symbols. These warnings are expected in the Docker/QEMU environment and do not necessarily indicate a problem. If GDB successfully connects and stops at main, you can continue.
- I created my repo but I can't find it

- Your repo is created under the cmpe120 organization. You can go to classroom50 and there will be a link to it if you lose track of it.
- Repository not found/permission denied
 - In your terminal run:
 - `gh auth login`
 - Answer the authentication questions as follows:

```
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 4219-DF77
Press Enter to open https://github.com/login/device in your browser...
```
 - Your one-time code will be different. Copy it.
 - Press enter and follow the instructions in your browser. Enter the one-time code when prompted.